

ĐẠI HỌC QUỐC GIA TP. HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC KHOA HỌC TỰ NHIÊN



Tài liệu thiết kế

Proof Of Concept
Tư Duy Tính Toán - CSC10014

Nhóm thực hiện:

Nhóm 4

Giảng viên hướng dẫn:

Nguyễn Trọng Việt

Thành phố Hồ Chí Minh, Ngày 22 tháng 7 năm 2026

DANH SÁCH THÀNH VIÊN

Link github đồ án: <https://github.com/TanKhoiTV/comptthink-2026>

MSSV	Họ và tên	Phân công công việc
24120010	Trần Văn Tấn Khôi	• Phác thảo sơ lược dàn ý POC • Phân công nội dung từng phần • Kiểm tra, kiểm chứng chất lượng nội dung • Lộ trình phát triển sản phẩm
24120017	Nguyễn Trọng An	• Tổng quan cấu trúc hệ thống • Chi tiết hạ tầng cho Frontend • Các quyết định kỹ thuật cho Frontend
24120028	Cao Việt Cường	• Chi tiết hạ tầng cho Backend • Các quyết định kỹ thuật cho Backend
24120035	Lê Thành Đạt	• Kiểm soát dữ liệu • Tổng hợp báo cáo, kiểm tra trình bày
24120036	Lê Văn Thành Đạt	• Luồng dữ liệu cho hệ thống
24120038	Nguyễn Phú Đạt	• Trải nghiệm của người dùng
24120138	Bùi Văn Thiên	• Vấn đề cần giải quyết • Các giới hạn và giả định trong đồ án • Đề xuất giải pháp
24120147	Văn Duy Thường	• Đối tượng mục tiêu và Use Case • Tính khả thi và rủi ro

MỤC LỤC

MỤC LỤC	ii
1 Vấn đề cần giải quyết	1
1.1 Thực trạng thị trường ứng dụng hỗ trợ du lịch	1
1.2 Khoảng trống của thị trường	1
1.3 Ý nghĩa	2
2 Giới hạn & Giả định	3
2.1 Giới hạn của dự án	3
2.2 Các giả định cốt lõi	3
2.3 Các phụ thuộc bên ngoài	4
3 Đề xuất giải pháp	5
3.1 Phân hệ Quản lý Tài nguyên & Kinh tế	5
3.2 Phân hệ Sa bàn & Quy hoạch Không gian	5
3.3 Mô phỏng & Kiểm chứng Logic	6
3.4 Ưu điểm	6
3.5 Các giới hạn của bản hiện tại (MVP)	7
4 Đối tượng mục tiêu & Use case	8
4.1 Đối tượng mục tiêu	8
4.1.1 Người dùng chính (Customer/Player)	8
4.1.2 Quản trị viên (Moderator)	8
4.1.3 Bảo trì viên (Maintainer)	9
4.1.4 Kiểm duyệt viên (Examiner)	9
4.1.5 Nhà cung cấp dữ liệu (Data Provider)	10
4.1.6 Phân tích viên (Analyst)	10

4.1.7	Content Manager/Service Provider	11
4.2	Các Use Cases nên tập trung cho MVP sắp tới	11
5	Kiến trúc	12
5.1	Tổng quan	12
5.1.1	Sơ đồ cấu trúc hệ thống	12
5.1.2	Ranh giới giữa Client và Server	13
5.1.3	Luồng tương tác giữa các vùng	15
5.2	Chi tiết hạ tầng	17
5.2.1	Frontend module	17
5.2.2	Backend module	19
5.3	Trải nghiệm người dùng & Luồng dữ liệu	21
5.3.1	Trải nghiệm người dùng - Quy trình hệ thống	21
5.3.2	Luồng dữ liệu	24
5.3.3	Luồng dữ liệu theo từng giai đoạn UI/UX	25
5.4	Kiểm soát dữ liệu	30
5.4.1	Dữ liệu lưu trữ dài hạn	30
5.4.2	Dữ liệu lưu trữ tạm thời	30
5.4.3	Chiến lược lưu trữ dữ liệu	31
5.4.4	Biện pháp bảo mật	31
5.5	Các kiểu dữ liệu chính (Core Types)	32
5.6	Môi trường phát triển & Triển khai	33
5.6.1	Công cụ phát triển	33
5.6.2	Môi trường triển khai	33
5.6.3	Quy trình build	34
6	Tính khả thi của đề án & Rủi ro có thể gặp	35
6.1	Tại sao dự án của nhóm có thể thực hiện được?	35

6.2	Rủi ro/rào cản có thể gặp phải và hướng xử lý	35
6.3	Kết luận	37
7	Lộ trình phát triển sản phẩm	39
7.1	MVP - Các chức năng cốt lõi & tiêu chí thành công	39
7.1.1	Gameplay cốt lõi	39
7.1.2	Nội dung & Dữ liệu	40
7.1.3	Hạ tầng kỹ thuật	40
7.1.4	Tiêu chí thành công của MVP	41
7.2	Bản phát hành chính thức	41
7.2.1	Tài khoản người chơi & Lịch sử	41
7.2.2	Tiến trình chiến dịch & Mở rộng bản đồ	42
7.2.3	Tính năng cộng đồng & Thi đua	42
7.2.4	Tiêu chí thành công của bản phát hành chính thức	42
8	Các quyết định kỹ thuật	43
8.1	Công nghệ thực hiện (Tech Stack)	43
8.1.1	Backend	43
8.1.2	Frontend	45
8.2	Các quyết định kiến trúc	46
8.2.1	Backend	46
8.2.2	Frontend	49

1. Vấn đề cần giải quyết

1.1. Thực trạng thị trường ứng dụng hỗ trợ du lịch

- **Nhóm Giao dịch & Tiện ích (OTAs & Navigation):** Các ứng dụng như Traveloka, Booking.com hay Google Maps giải quyết rất tốt bài toán đơn lẻ như đặt phòng, mua vé, tìm đường từ điểm A đến điểm B.
- **Nhóm Tổng hợp thông tin (Aggregators & Reviews):** Các nền tảng như TripAdvisor hay các blog du lịch cung cấp kho dữ liệu khổng lồ về các địa điểm tham quan, quán ăn, và đánh giá của người dùng hoặc tính toán và gợi ý lịch trình cho khách hàng.

1.2. Khoảng trống của thị trường

Mặc dù có vô vàn công cụ du lịch và hàng ngàn tựa game trên thị trường, nhưng đang tồn tại một sự thiếu sót lớn sự giao thoa giữa Công cụ thực dụng và Trải nghiệm giải trí. Cụ thể:

Sự thiếu tính kích thích, thụ động và rập khuôn của các công cụ hỗ trợ lên lịch trình hay lên Lịch trình Tự động: Các ứng dụng du lịch hiện tại biến việc lên lịch trình thành một việc mệt mỏi, nơi người dùng phải tự làm tính toán để cân đối ngân sách, khoảng cách và thời gian. Mặc dù hiện nay, nhiều ứng dụng đã tích hợp AI hoặc tính năng tự động gợi ý lịch trình và chi phí. Tuy nhiên, chúng lại rơi vào một điểm yếu chí mạng: Biến người dùng thành người tiếp nhận thụ động. Các lịch trình tạo sẵn thường mang tính rập khuôn, cứng nhắc. Việc phó mặc hoàn toàn cho hệ thống vô tình tước đoạt đi niềm vui tìm tòi, sự hào hứng và tính cá nhân hóa. Người dùng không có sự gắn kết cảm xúc hay cảm giác sở hữu đối với một lịch trình do máy tự động tạo ra.

Sự vắng bóng của mô hình "Play to Plan": Trái ngược với sự khô khan của app du lịch, các tựa game mô phỏng/chiến thuật mang lại sự tương tác và hứng thú cực cao, nhưng lại hoàn toàn ảo, chơi xong không đọng lại giá trị thực tiễn nào. Thị trường đang vắng bóng một sản phẩm giao thoa giữa cả hai làm cầu nối: Vừa trao quyền để người dùng tự tay nhào nặn chuyến đi thông qua các quyết định chiến thuật giải trí, vừa tạo ra được một kết quả có thể ứng dụng ngay vào đời thực

Thiếu môi trường thử và sai trực quan trước chuyến đi: Kể cả khi dùng app tự động lên lịch, du khách vẫn không thể hình dung được mức độ hợp lý của nó cho đến khi thực sự trải

nghiệm. Không có một hệ thống nào cho phép người dùng đi thử lịch trình đó, đưa ra các biến số ngẫu nhiên (kẹt xe, thời tiết) để chấm điểm và cảnh báo mức độ rủi ro một cách sinh động, trực quan trước khi dùng lịch trình đó cho việc du lịch thực tế.

1.3. Ý nghĩa

- **Mô hình Chơi Game Ảo - Lịch Trình Thật:** Đây là giá trị cốt lõi của dự án. Người dùng sử dụng hệ thống với tâm thế tận hưởng một Board Game chiến thuật (mua bán thẻ bài, xếp sa bàn lưới, đua top điểm). Tuy nhiên, ngay khi ván game kết thúc, Bàn cờ mà họ vừa tối ưu hóa đó chính là một **Lịch trình du lịch có tính khả thi cao**, đã tối ưu hóa về ngân sách, sức khỏe và trình tự di chuyển để ứng dụng trực tiếp vào đời thực.
- **Kênh quảng bá và kích cầu du lịch tương tác cao:** Trò chơi biến mỗi địa danh, mỗi món ăn bản địa thành một Thẻ bài có các thông tin và thông số riêng. Nhờ đó, game vô tình trở thành một nền tảng quảng bá văn hóa nước nhà một cách tự nhiên. Thay vì tiếp nhận thông tin thụ động, trò chơi buộc người dùng nghiên cứu sâu các thuộc tính, đặc điểm của từng địa danh được số hóa dưới dạng **Thẻ bài** để xây dựng chiến thuật. Việc lồng ghép ẩm thực, văn hóa vào cơ chế tính điểm giúp người chơi ghi nhớ và nảy sinh nhu cầu khám phá thực tế một cách tự nhiên, biến trò chơi thành công cụ kích cầu du lịch chiều sâu.

2. Giới hạn & Giả định

2.1. Giới hạn của dự án

- **Về mô hình vận hành:** Trò chơi là một hệ thống Mô phỏng khép kín, loại bỏ hoàn toàn sự phụ thuộc vào các API. Việc tính toán khoảng cách và logic di chuyển được thực hiện thông qua **Tọa độ mô phỏng** lưu trữ tĩnh trong thẻ bài. Điều này giúp hệ thống hoạt động ổn định, bảo mật quyền riêng tư tuyệt đối và cho phép người dùng trải nghiệm ở bất cứ đâu.
- **Về phạm vi nội dung & Dữ liệu (Content Scope):** Trong giai đoạn POC, hệ thống chỉ tập trung vào dữ liệu của Phase 1 Sài Gòn.
- **Về chế độ kết nối (Connectivity & Multiplayer):** Chế độ nhiều người chơi trực tuyến đã được tích hợp qua Socket.IO (WebSocket). Chế độ Chơi đơn sử dụng cùng Room FSM với 3 bot đối thủ chạy trên server (localRoom.ts). Chế độ Hotseat (Đối kháng cục bộ) chưa được triển khai.
- **Về môi trường thực thi (Platform):** Dự án triển khai trên nền tảng Web theo hướng **Mobile-first PWA**. Việc đóng gói thành ứng dụng Native (Android/iOS) qua App Store/Play Store nằm ngoài phạm vi.

2.2. Các giả định cốt lõi

Sự vận hành của mô hình "Play to Plan" dựa trên các giả định nền tảng sau:

- **Hành vi & Nhận thức người dùng:** Giả định người chơi hứng thú và sẵn lòng tham gia vào các thử thách quản lý tài nguyên (Xu, Thẻ lực) để tối ưu hóa điểm số. Như một phần của thử thách trí tuệ từ đó gián tiếp tạo ra một lịch trình du lịch cá nhân hóa.
- **Dữ liệu:** Giả định bộ dữ liệu về tọa độ (Lat/Lng) tĩnh và các đặc tính văn hóa của thẻ bài là chính xác so với thực địa, đảm bảo kết quả lịch trình xuất ra sau ván game mang giá trị ứng dụng cao.
- **Hiệu suất:** Giả định framework Frontend được chọn có khả năng xử lý mượt mà các trạng thái phức tạp của sa bàn lịch trình cùng các hiệu ứng tương tác kéo thả trên các thiết bị

di động tầm trung.

2.3. Các phụ thuộc bên ngoài

- **Nền tảng lưu trữ:** Hệ thống phụ thuộc vào sự ổn định của nền tảng Backend trong việc cung cấp dữ liệu thẻ bài và lưu trữ tiến trình người chơi.
- **Thư viện Thuật toán & Giao diện:** Sử dụng các mã nguồn mở để đảm bảo tính chuẩn xác trong đo đạc khoảng cách giữa các tọa độ ảo, cũng như các thư viện hỗ trợ Drag & Drop trên Web để hiện thực hóa trải nghiệm xếp sa bàn lịch trình.

3. Đề xuất giải pháp

Giải pháp trọng tâm của dự án là xây dựng hệ thống là một nền tảng game chiến thuật (Strategy Board Game) giả lập. Giải pháp này không chỉ đóng vai trò là một trò chơi giải trí mà còn là công cụ để **Tối ưu hóa Lịch trình** dựa trên dữ liệu thực tế. Cấu trúc giải pháp được chia thành 3 phân hệ logic chính:

3.1. Phân hệ Quản lý Tài nguyên & Kinh tế

Hệ thống thiết lập một môi trường quản lý tài nguyên nhằm thử thách khả năng ra quyết định của người chơi:

- **Hệ thống Tài nguyên Kép:** Người chơi phải cân đối giữa Xu (Ngân sách) và Lá (Thẻ lực). Mọi thẻ bài địa điểm đều yêu cầu mức chi trả khác nhau cho hai loại tài nguyên này.
- **Ràng buộc Sức chứa:** Túi đồ của người chơi có giới hạn vật lý. Điều này buộc người chơi phải thực hiện các phép tính toán chi phí cơ hội: Mua thẻ giá trị cao ngay lập tức hay tích trữ tài nguyên cho vòng sau.
- **Cơ chế Đánh đổi:** Hệ thống tích hợp tính năng "Vay nợ" hoặc "Vắt kiệt thẻ lực". Người chơi có thể vượt rào tài nguyên để lấy thẻ quan trọng, nhưng hệ thống sẽ tự động gán các "Debuff" như trừ điểm hoặc khóa các ô thời gian ở ngày tiếp theo (giả lập trạng thái kiệt sức/ thiếu tiền).

3.2. Phân hệ Sa bàn & Quy hoạch Không gian

Hệ thống sử dụng một ma trận lưới 5×5 đại diện cho 5 ngày du lịch, mỗi ngày gồm 5 khung giờ (Sáng, Trưa, Chiều, Tối, Khuya).

- **Cấu trúc Ma trận:** Người chơi thực hiện thao tác kéo-thả (Drag & Drop) để lắp ghép các thẻ bài vào lưới.
- **Thuật toán Duyệt mảng & Liên kết:** Hệ thống tự động quét các ô theo trục dọc và ngang để phát hiện các nhóm thẻ có cùng thuộc tính (Tags). Khi người dùng thiết lập được các chuỗi địa điểm cùng bộ, hệ thống sẽ kích hoạt Hệ số nhân điểm thưởng, khuyến

khích tư duy quy hoạch theo cụm.

- **Chiến thuật Khoảng đệm:** Khác với các app tự động, giải pháp cho phép người chơi chủ động để trống các ô thời gian. Thuật toán sẽ nhận diện các ô trống này là Thời gian di chuyển/Nghỉ ngơi, giúp hóa giải các hình phạt về khoảng cách địa lý giữa hai địa điểm nằm xa nhau trên bản đồ.

3.3. Mô phỏng & Kiểm chứng Logic

Kiểm chứng Khoảng cách - Sử dụng công thức để tính toán khoảng cách thực tế giữa các tọa độ của các thẻ bài

Nếu người chơi xếp hai địa điểm có khoảng cách vật lý quá xa trong hai khung giờ liên tiếp, hệ thống sẽ tự động giáng hình phạt trừ điểm, mô phỏng sự lãng phí thời gian và mệt mỏi khi di chuyển.

- **Xác suất Sự kiện ngẫu nhiên:** Hệ thống tích hợp cơ chế tạo sự kiện ngẫu nhiên dựa trên các tag của thẻ bài, mô phỏng các sự kiện ngoài đời thật. Cơ chế này buộc người chơi phải có các phương án dự phòng.
- **Chuyển đổi Kết quả:** Sau khi hoàn thành phần chơi, bàn chơi sẽ được hệ thống xuất ra dưới dạng một **Bản kế hoạch du lịch thực tế**. Đây là bước chuyển đổi cốt lõi từ "Game ảo" sang "Ứng dụng thật", giúp người dùng sở hữu một lịch trình đã được kiểm chứng về mặt logic và kinh tế.

3.4. Ưu điểm

- **Sự chủ động:** Thay vì nhận một lịch trình áp đặt từ AI, người chơi được trực tiếp nhào nặn chuyến đi của mình thông qua tư duy chiến thuật, tạo ra sự gắn kết với kế hoạch đã lập.
- **Môi trường Sandbox an toàn:** Người dùng có thể thử nghiệm những lịch trình rủi ro cao và nhìn thấy kết quả mô phỏng ngay lập tức mà không phải đánh đổi bằng tiền bạc hay sức khỏe ngoài đời thực.
- **Tính trực quan:** Việc số hóa các địa danh thành thẻ bài với thông số cụ thể giúp người

dùng dễ dàng so sánh, lựa chọn và ghi nhớ thông tin văn hóa một cách tự nhiên thông qua quá trình chơi.

3.5. Các giới hạn của bản hiện tại (MVP)

Bản triển khai hiện tại có các giới hạn sau, được xác định dựa trên phạm vi của đề án:

- **Multiplayer:** Hỗ trợ matchmaking tự động qua hàng đợi, bot fill sau 30s để đủ 4 người. Chưa có chế độ mời bạn bè hoặc phòng riêng.
- **Campaign:** Chỉ gồm Phase 1 (Sài Gòn) với 100+ thẻ địa điểm. Các nhánh bản đồ Đà Lạt, Đà Nẵng, Nha Trang, Huế chưa được triển khai.
- **Lưu trữ:** Sử dụng PostgreSQL (`server/db.ts`) với các bảng `users`, `matches`, `match_players`. Server vẫn chạy được ở chế độ không-DB nếu chưa cấu hình `DATABASE_URL`. Chưa có migration tự động hoặc rollback.
- **Xác thực:** Hỗ trợ đăng ký/đăng nhập bằng email/mật khẩu với PBKDF2 + JWT (`server/auth.ts`). Chưa có OAuth, xác thực hai yếu tố hoặc đăng nhập qua mạng xã hội.
- **Leaderboard:** Hiện thị điểm VP trong phòng chơi, chưa có bảng xếp hạng toàn cầu hoặc giữa các nhóm bạn bè.
- **Admin panel:** Chưa có giao diện quản trị nội dung. Việc thêm/sửa thẻ địa điểm phải thực hiện thủ công qua file nguồn (`src/data/cards.phase1.ts`).
- **Xuất lịch trình:** Chỉ xuất được timeline dạng văn bản trong game, chưa có xuất file PDF hoặc chia sẻ qua link.

4. Đối tượng mục tiêu & Use case

4.1. Đối tượng mục tiêu

4.1.1. Người dùng chính (Customer/Player)

Chơi game để nhận lịch trình du lịch: Người dùng sẽ tham gia vào một trò chơi mô phỏng việc lập lịch trình du lịch. Bắt đầu màn chơi, người dùng sẽ lựa chọn và sắp xếp các Thẻ Địa Điểm vào Bàn Cờ Lịch Trình theo các mốc thời gian trong ngày. Trong quá trình chơi, người dùng cần quản lý tài nguyên như Xu Vàng và Thẻ lực, cân nhắc khoảng cách giữa các địa điểm, tận dụng combo và xử lý các sự kiện ngẫu nhiên để đạt được điểm Hạnh Phúc VP cao nhất. Sau khi hoàn thành màn chơi, hệ thống sẽ tổng hợp các thẻ đã được sắp xếp thành một lịch trình du lịch hoàn chỉnh. Người dùng có thể dùng nó làm gợi ý để áp dụng vào chuyến đi ngoài đời.

Quản lý tài khoản và hành trang: Người dùng có thể xem lại lịch sử màn chơi, điểm số, lịch trình đã tạo, thành tựu và các vật phẩm trong game nếu có.

Đánh giá và chia sẻ trải nghiệm: Người dùng có thể chia sẻ lịch trình đã tạo ra sau khi hoàn thành màn chơi, bao gồm điểm VP, các địa điểm đã chọn, combo đạt được và lịch trình theo từng ngày. Use case này giúp tăng tính tương tác cộng đồng và cho phép người chơi tham khảo lịch trình của nhau.

4.1.2. Quản trị viên (Moderator)

Tinh chỉnh nội dung game: Moderator có thể thêm, sửa hoặc xóa các địa điểm, thẻ bài,... Nội dung game phải được cập nhật thường xuyên để tránh nhàm chán và để đảm bảo nhiệm vụ vẫn phù hợp với thực tế.

Quản lý session của người chơi: Mỗi session lưu trạng thái tiến trình của người chơi, bao gồm các thẻ đã chọn, tài nguyên hiện có, vị trí thẻ trên bàn cờ lịch trình, điểm VP, combo và lịch trình đầu ra. Nếu session bị lỗi, người chơi có thể mất tiến trình, sai lệch điểm số hoặc tạo ra lịch trình không hợp lệ. Vì vậy, hệ thống cần cơ chế phân quyền rõ ràng, ghi log thao tác và hạn chế chỉnh sửa trực tiếp vào session đang hoạt động.

Quản lý người dùng: Moderator cần khả năng xem, khóa hoặc xử lý tài khoản trong các trường hợp vi phạm, gian lận hoặc có yêu cầu hỗ trợ.

4.1.3. Bảo trì viên (Maintainer)

Xử lý lỗi hệ thống: Khi server, API hoặc database gặp lỗi, các tính năng như tải dữ liệu địa điểm, đồng bộ điểm số, quản lý tài khoản hoặc cập nhật nội dung đều có thể bị ảnh hưởng. Do đó, hệ thống cần có log đủ chi tiết, cơ chế cảnh báo và quy trình xử lý lỗi rõ ràng.

Cập nhật hệ thống: Maintainer triển khai tính năng mới, sửa lỗi và rollback khi bản cập nhật gây lỗi. Với một ứng dụng có logic game và dữ liệu địa điểm, thẻ bài thay đổi liên tục, việc cập nhật phải tránh làm hỏng trạng thái người chơi hoặc dữ liệu hiện có.

Quản lý dữ liệu: Bao gồm backup, restore và bảo vệ dữ liệu. Đây là use case quan trọng vì hệ thống có thể lưu lịch trạng thái ván chơi, lịch sử điểm số, dữ liệu thẻ bài, lịch trình đã tạo và thông tin tài khoản.

4.1.4. Kiểm duyệt viên (Examiner)

Đánh giá tuân thủ quyền riêng tư và thu thập dữ liệu: Ứng dụng cần giải thích rõ lý do xin quyền vị trí, camera hoặc dữ liệu liên quan đến hoạt động người dùng. Việc xin quyền phải minh bạch, đúng ngữ cảnh và không gây cảm giác bị ép buộc. Nếu luồng xin quyền không rõ ràng, người dùng có thể từ chối cấp quyền, làm gián đoạn các tính năng cốt lõi như check-in hoặc xác thực nhiệm vụ.

Kiểm định phân loại độ tuổi và an toàn cho trẻ em: Do ứng dụng có yếu tố game, huy hiệu và phần thưởng, sản phẩm có thể thu hút người dùng nhỏ tuổi. Vì vậy, hệ thống cần tránh đề xuất địa điểm không phù hợp, hạn chế chia sẻ dữ liệu nhạy cảm và có cơ chế bảo vệ nhóm người dùng dễ bị tổn thương.

Rà soát bản quyền và sở hữu trí tuệ: Examiner cần kiểm tra tài nguyên hình ảnh, icon, vector art, tên thương hiệu và nội dung địa điểm để tránh sử dụng sai giấy phép hoặc vi phạm nhãn hiệu.

Kiểm định tuân thủ luật bảo vệ dữ liệu: Hệ thống cần hỗ trợ các nguyên tắc như chỉ thu thập dữ liệu cần thiết, cho phép người dùng xóa tài khoản/dữ liệu, và ưu tiên xử lý dữ liệu theo hướng bảo vệ quyền riêng tư.

4.1.5. Nhà cung cấp dữ liệu (Data Provider)

Cung cấp dữ liệu bản đồ nền nếu cần: Bản đồ có thể được dùng để minh họa tuyến du lịch hoặc hỗ trợ hiển thị lịch trình cuối game.

Xác thực vị trí: Khi người dùng check-in, hệ thống cần đối chiếu tọa độ thiết bị với tọa độ địa điểm trên thẻ bài.

Cung cấp ngữ cảnh thời gian thực: Dữ liệu thời tiết hoặc giao thông có thể được dùng để điều chỉnh nhiệm vụ, ví dụ giảm ưu tiên nhiệm vụ ngoài trời khi trời mưa hoặc thay đổi độ khó khi giao thông xấu.

4.1.6. Phân tích viên (Analyst)

Phân tích cân bằng tài nguyên và độ khó của game: Analyst theo dõi cách người chơi sử dụng xu vàng, thẻ lực, cơ chế vay xu và vay thẻ lực để đánh giá trò chơi có quá dễ, quá khó hoặc gây phạt quá nặng hay không, từ đó điều chỉnh giới hạn tài nguyên, chi phí thẻ, mức phạt nợ và hình phạt khi người chơi vắt kiệt sức.

Đánh giá chất lượng lịch trình được tạo ra sau mỗi ván chơi: Sau khi người chơi hoàn thành game, hệ thống tạo ra một lịch trình du lịch dựa trên các Thẻ địa điểm đã được xếp vào bàn cờ. Analyst cần đánh giá liệu lịch trình này có hợp lý để áp dụng ngoài đời hay không, ví dụ: các địa điểm có quá xa nhau, mốc thời gian có phù hợp, lịch trình có cân bằng giữa ăn uống, tham quan, nghỉ ngơi và giải trí hay không.

Phân tích hiệu quả của thẻ, combo và sự kiện ngẫu nhiên: Analyst theo dõi những nhóm thẻ nào được chọn nhiều, combo nào quá mạnh hoặc quá yếu, sự kiện nào thường gây bất lợi lớn cho người chơi. Từ đó, nhóm có thể đề xuất điều chỉnh điểm VP, hiệu ứng thẻ, xác suất sự kiện và điều kiện kích hoạt để gameplay công bằng và thú vị hơn.

Phát hiện chiến thuật lặp lại hoặc mất cân bằng trong cách chơi: Nếu đa số người chơi luôn chọn cùng một loại thẻ, cùng một tuyến du lịch hoặc cùng một chiến thuật để đạt điểm cao, điều đó cho thấy game có thể đang thiếu đa dạng. Analyst cần phát hiện các mẫu hành vi này để hỗ trợ Game Designer điều chỉnh hệ thống, khuyến khích người chơi thử nhiều cách xây dựng lịch trình khác nhau.

Đánh giá khả năng chuyển đổi từ game sang lịch trình thực tế: Analyst có thể so sánh

lịch trình được tạo ra với các tiêu chí cơ bản như khoảng cách di chuyển, độ đa dạng hoạt động, số lượng địa điểm mỗi ngày và mức độ phù hợp cho hoạt động du lịch. Đây là use case quan trọng vì sản phẩm không chỉ là một trò chơi, mà còn phải tạo ra đầu ra có ích cho chuyến đi ngoài đời.

4.1.7. Content Manager/Service Provider

Quản lý dữ liệu địa điểm và thẻ bài: Content Manager cập nhật thông tin cho các Thẻ Địa Điểm như tên địa điểm, mô tả, loại hình hoạt động, tag, vị trí, đặc điểm nổi bật và mức độ phù hợp với từng mốc thời gian trong ngày. Use case này rất quan trọng vì mỗi thẻ không chỉ là một thành phần trong game, mà còn đại diện cho một địa điểm thực tế trong lịch trình cuối cùng.

Thiết lập chỉ số gameplay cho thẻ địa điểm: Content Manager phối hợp với Game Designer để gán các chỉ số như điểm VP cơ bản, chi phí xu vàng, mức tiêu hao thẻ lực, nhóm thẻ và hiệu ứng đặc biệt. Việc thiết lập này ảnh hưởng trực tiếp đến độ cân bằng của trò chơi, vì nếu một nhóm thẻ quá mạnh hoặc quá rẻ, người chơi sẽ có xu hướng lặp lại cùng một chiến thuật thay vì xây dựng lịch trình đa dạng.

Quản lý ưu đãi hoặc nội dung đối tác (nếu có): Service Provider có thể tham gia bằng cách cung cấp thông tin địa điểm, ưu đãi hoặc nội dung quảng bá.

4.2. Các Use Cases nên tập trung cho MVP sắp tới

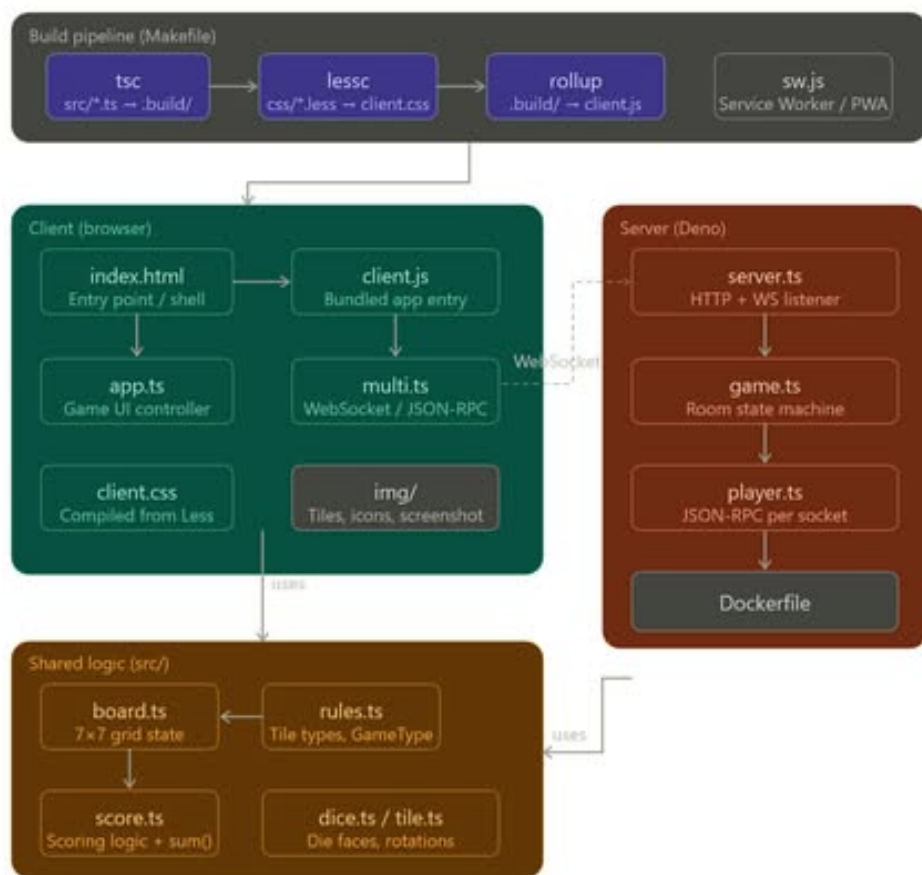
- Customer chơi được một màn game lập lịch trình du lịch ảo.
- Hệ thống có bộ Thẻ Địa Điểm với tag, vị trí, chi phí tài nguyên và điểm VP.
- Người dùng có thể quản lý xu vàng, thẻ lực và xếp thẻ vào bàn cờ lịch trình.
- Hệ thống có thể tính điểm VP dựa trên combo, tài nguyên, khoảng cách và sự kiện.
- Sau ván chơi, hệ thống tạo được lịch trình du lịch hoàn chỉnh để tham khảo ngoài đời.
- Trạng thái ván chơi, điểm số và lịch trình đầu ra được lưu ổn định.
- Content Manager/Moderator chỉnh được dữ liệu thẻ và chỉ số gameplay cơ bản.
- Analyst có thể theo dõi dữ liệu playtest để cân bằng game và đánh giá chất lượng lịch trình.

5. Kiến trúc

5.1. Tổng quan

Hệ thống được thiết kế theo kiến trúc **Stateful Real-time PWA**, tối ưu hóa cho trải nghiệm di động và khả năng chơi offline. Kiến trúc này không chia cắt cứng nhắc giữa các tầng mà kết nối chặt chẽ thông qua một lõi logic dùng chung (Shared Logic), đảm bảo tính đồng nhất giữa trải nghiệm người dùng và quy tắc nghiệp vụ của máy chủ.

5.1.1. Sơ đồ cấu trúc hệ thống



Hình 5.1: Kiến trúc tổng quát

Tổ chức hệ thống được phân tách thành 3 phân vùng (Panels) chức năng chính, xoay quanh một lõi domain tập trung:

- Build Pipeline (Hạ tầng sản xuất - Makefile): Chuyển đổi mã nguồn (`.ts`, `.less`) thành các tài nguyên phân phối (`.js`, `.css`).
- Server Side (Hệ thống máy chủ - Node.js + tsx): Quản lý quyền lực (Authority), trạng thái phòng chơi (Room State), và kết nối đa người chơi qua Socket.IO.
- Client Side (Giao diện người dùng - Browser): Render giao diện đồ họa, xử lý tương tác trực tiếp và phản hồi tức thời (Optimistic UI).
- Shared Logic (Nhân Domain - `src/`): Chứa bộ luật chơi (`rules.ts`, `score.ts`). Đây là thành phần duy nhất chạy song song ở cả Client và Server để đảm bảo tính nhất quán.

5.1.2. Ranh giới giữa Client và Server

Ranh giới giữa Frontend và Backend hoạt động theo cơ chế State Synchronization (Đồng bộ trạng thái) thay vì các Request/Response tĩnh thông thường.

Giao thức kết nối

- Kênh truyền dẫn: Socket.IO (WebSocket với fallback).
- Định dạng thông điệp: Socket.IO events. Máy chủ định nghĩa kiểu sự kiện `ClientToServerEvents` và `ServerToClientEvents` trong file `types.ts`, đảm bảo an toàn kiểu dữ liệu giữa client và server. Các module xử lý bao gồm: `rooms.ts` (lifecycle phòng), `draftEngine.ts` (draft), `timerEngine.ts` (simulation).

Các sự kiện Socket.IO chính

Loại	Tên sự kiện	Mô tả
C→S	<code>room:create</code>	Tạo phòng mới và vào game ngay, có thể chọn thành phố/tutorial
C→S	<code>room:join</code>	Tham gia phòng theo roomId
C→S	<code>room:reconnect</code>	Kết nối lại phòng khi mất mạng
C→S	<code>room:leave</code>	Rời phòng chơi
C→S	<code>room:setReady</code>	Đánh dấu sẵn sàng

C→S	game:start	Bắt đầu game (sau khi tất cả sẵn sàng)
C→S	matchmaking:find	Vào hàng đợi tìm trận tự động
C→S	matchmaking:cancel	Hủy tìm trận
C→S	draft:selectCard	Chọn thẻ trong pool draft
C→S	draft:confirmPick	Xác nhận giữ thẻ và chuyển phần còn lại
C→S	planning:placeCard	Đặt thẻ vào ô (rowIndex, colIndex)
C→S	planning:discardCard	Hủy thẻ để thu hồi tài nguyên
C→S	planning:payDebt	Trả Token Nợ bằng tài nguyên
C→S	planning:returnBoardCard	Gỡ thẻ khỏi board về lại hand
C→S	planning:confirm	Chốt lịch trình ngày hiện tại
C→S	tutorial:pauseReplay	Tạm dừng replay trong tutorial
C→S	tutorial:resumeReplay	Tiếp tục replay trong tutorial
S→C	room:joined	Đã vào phòng (kèm roomId, playerId, state)
S→C	room:state	Snapshot trạng thái phòng theo từng người chơi
S→C	room:left	Đã rời phòng
S→C	matchmaking:status	Số người đang chờ trong hàng đợi (count, target)
S→C	game:error	Thông báo lỗi (hành động không hợp lệ)

Phân chia trách nhiệm

Hệ thống vận hành theo nguyên lý "Optimistic Client — Authoritative Server":

Đặc điểm	Phía Client (Frontend)	Phía Server (Backend)
Vai trò chính	Trải nghiệm & Tương tác (UX)	Quyền quyết định & Sự thật (Authority)
Xử lý Logic	Dự đoán kết quả (Preview) dựa trên Shared Logic để phản hồi UI tức thì.	Tái kiểm tra (Validate) mọi hành động gửi lên để ngăn chặn gian lận.
Trạng thái	Chỉ lưu giữ trạng thái cục bộ của 1 người dùng.	Quản lý trạng thái tổng thể của cả phòng chơi (Room FSM).
Lưu trữ	Caching tài nguyên qua Service Worker.	Lưu trữ trạng thái trận đấu trong bộ nhớ Node.js (In-memory).

Bảng 5.2: Phân chia trách nhiệm giữa Client và Server

5.1.3. Luồng tương tác giữa các vùng

Để minh họa ranh giới này, xét quy trình khi một người chơi thực hiện nước đi:

- Tại Client: `app.ts` gọi `rules.ts` (Shared) để kiểm tra hợp lệ ngay lập tức và hiển thị điểm dự kiến (0ms delay).
- Qua Boundary: `socketClient.ts` gửi gói tin Socket.IO event (`socket.emit`) qua Web-Socket.
- Tại Server: `rooms.ts` tiếp nhận, `gameEngine.ts` gọi chính `rules.ts`, `board.ts`, `score.ts` (Shared) để xác thực lần cuối.
- Đồng bộ: Nếu hợp lệ, Server cập nhật trạng thái chung và broadcast snapshot mới nhất cho tất cả người chơi để đồng bộ hóa giao diện.

Kiến trúc này giúp dự án đạt được sự cân bằng giữa tốc độ phản hồi của ứng dụng Offline và tính bảo mật, đồng bộ của ứng dụng Online.

5.2. Chi tiết hạ tầng

5.2.1. Frontend module

Build Pipeline

Module	Loại	Sở hữu	Không sở hữu
<code>tsc</code>	Trình biên dịch	Chuyển đổi TS → JS, kiểm tra lỗi kiểu dữ liệu (type checking)	Đóng gói tệp cuối, quản lý tài nguyên tĩnh
<code>lessc</code>	Trình tiền xử lý	Chuyển đổi Less → CSS, xử lý biến và hàm giao diện	Thực thi logic game, đóng gói tệp JS
<code>build/</code>	Kho lưu trữ	Lưu trữ các tệp <code>.js</code> và <code>.css</code> đã biên dịch	Mã nguồn gốc (<code>src</code>), tệp đóng gói hoàn chỉnh
<code>sw.js</code>	Trình trung gian	Chiến lược lưu đệm (caching), hỗ trợ chơi of- fline	Render giao diện, logic phía server

Bảng 5.3: Build Pipeline

Ghi chú: Dự án không sử dụng bundler. Đầu ra của `tsc` là các tệp `.js` riêng lẻ trong `build/`, không gom thành `client.js` duy nhất.

Client/Browser

Module	Loại	Sở hữu	Không sở hữu
index.html	Điểm nhập	Cấu trúc DOM, khai báo nạp tài nguyên hệ thống	Logic xử lý, các quy tắc định dạng style
client.css	Tệp định dạng	Các quy tắc CSS, bố cục (layout), hiệu ứng chuyển động	Logic thành phần, quản lý tài nguyên ảnh
app.ts	Bộ điều phối UI	Quản lý trạng thái view, xử lý sự kiện (event), DOM, render, drag-and-drop	Đồng bộ đa người chơi
socketClient.ts	Giao diện mạng	Vòng đời Socket.IO, kết nối phòng, gửi/nhận sự kiện RPC, xác thực token	Cơ chế gameplay, render đồ họa cục bộ
assets/	Tài nguyên tĩnh	Hình ảnh icon, ảnh nền, tài nguyên đồ họa	Mã nguồn thực thi, dữ liệu trạng thái

Bảng 5.4: Client / Browser

5.2.2. Backend module

Module	Sở hữu	Không sở hữu
<code>index.ts</code>	HTTP listener + Socket.IO, /health endpoint, socket binding, CORS	Business logic, trạng thái người chơi
<code>rooms.ts</code>	Room lifecycle (create, join, leave, reconnect), quản lý phase, xác thực người chơi, broadcast snapshot	Chi tiết game engine, draft engine
<code>gameEngine.ts</code>	Game state factories (board, deck, player, view); load card data (phase1-3)	Xử lý draft rotation, timer simulation
<code>draftEngine.ts</code>	Draft state machine: pool creation, rotation, pick/confirm	Render UI, tính điểm
<code>timerEngine.ts</code>	Simulation tick: debt scan, random events, combo, distance, day transitions	Quản lý phòng, xác thực
<code>auth.ts</code>	PBKDF2 + JWT, register/login/verify qua PostgreSQL (server/db.ts)	Gameplay logic

Bảng 5.5: Module/service của Backend (Phần 1: Core Engine & Network)

Module	Sở hữu	Không sở hữu
bot.ts	Bot AI: fillRoomWithBots, auto pick/draft/place/confirm (greedy)	Logic game, render UI
db.ts	PostgreSQL: bảng users, matches, match_players; initDb, saveMatchResult	Gameplay logic, render UI
types.ts	Socket.IO wire event types (ClientToServerEvents, Server- ToClientEvents)	Business logic

Bảng 5.6: Module/service của Backend (Phần 2: Auth, Database & Types)

5.3. Trải nghiệm người dùng & Luồng dữ liệu

5.3.1. Trải nghiệm người dùng - Quy trình hệ thống

0. Giai đoạn Matchmaking & Cinematic

Mô tả:

- **Hàng đợi Matchmaking:** Người chơi chọn thành phố (Sài Gòn, Đà Nẵng, Hà Nội) qua màn hình `mapSelection.ts`, sau đó vào hàng đợi tìm trận. Hệ thống ghép tối đa 4 người thật; nếu sau 30s chưa đủ, bot sẽ được thêm vào cho đủ 4 người (`fillRoomWithBots` trong `server/bot.ts`).
- **Cinematic (Boarding Gate):** Sau khi ghép trận thành công, phòng chuyển sang phase "cinematic". Màn hình boarding gate hiển thị video máy bay cất cánh, đếm ngược 7 giây, sau đó bắt đầu gameplay.
- **Chơi ngay (tạo phòng riêng):** Người chơi có thể tạo phòng riêng (`room:create`) để vào game ngay mà không chờ matchmaking. Bot tự động điền vào các ghế trống nhờ `fillRoomWithBots`.

A. Giai đoạn Drafting (Daily Drafting Phase)

Mô tả:

- **Cơ chế phát thẻ:** Mỗi ngày, hệ thống chia 7 thẻ cho mỗi người chơi (`DRAFT_STARTING_POOL_SIZE = 7`). Thực hiện 5 lượt chọn-giữ-chuyển (`HAND_SIZE = 5`): mỗi lượt người chơi chọn giữ 1 thẻ và chuyển các thẻ còn lại theo chiều kim đồng hồ. Bot tự động pick thẻ VP cao nhất (greedy strategy) qua `server/bot.ts`. Deck được tạo cân bằng giữa 4 nhóm nhờ `createBalancedRandomDeck` trong `src/game/draft.ts`.
- **Cơ chế Vay nợ (Debt Mechanism):** Khi thiếu Xu, người chơi vẫn có thể lấy thẻ nhưng hệ thống sẽ gán "Token Nợ". Token này dùng để khấu trừ điểm ở bước tính toán cuối cùng.
- **Cơ chế Kiệt sức (Exhaustion):** Nếu sử dụng thẻ khi cạn Thẻ lực, hệ thống tự động khóa (Lock) một ô thời gian ngẫu nhiên của ngày kế tiếp, vô hiệu hóa thao tác tại ô đó.
- **Quy đổi tài nguyên (Discard):** Người chơi có quyền hủy thẻ để lấy lại một lượng Xu/Thẻ

lực nhất định. Ô trống sau khi hủy thẻ được tính là “Thời gian nghỉ”, giúp ngắt các ràng buộc về phạt khoảng cách.

B. Giai đoạn Lên lịch trình

Mô tả:

- **Phân bổ Grid:** Người dùng thực hiện chọn và kéo thả thẻ bài vào các slot Sáng - Trưa - Chiều - Tối - Khuya trên Grid 5×5 (5 ngày $\times$ 5 khung giờ).
- **Chốt ngày:** Sau khi xác nhận lịch trình trong ngày, các thẻ chưa sử dụng trong kho sẽ bị hủy để giải phóng bộ nhớ cho ngày làm việc tiếp theo.

C. Giai đoạn Simulation & Scoring

Mô tả: Sau mỗi ngày (khi người chơi xác nhận planning), hệ thống khóa Grid của ngày đó và thực hiện chuỗi thuật toán tính điểm tự động gồm 5 bước. Vòng lặp này lặp lại cho tất cả 5 ngày. Sau ngày thứ 5, hệ thống chuyển sang phase "result" hiển thị các bước replay (từng sự kiện, combo, khoảng cách) rồi mới đến màn hình Game Over với tổng kết điểm VP cuối cùng.

- **Bước 1 (Check Nợ):** Kiểm tra số lượng Token Nợ và thực hiện trừ điểm tương ứng.
- **Bước 2 (Random Events):** Chạy xác suất 15% kích hoạt sự kiện dựa trên tag của thẻ (thời tiết, giao diện, sức khỏe). Kết quả được tính bằng hàm hash có seed (không dùng Math.random), đảm bảo tái tạo được giống hệt khi cùng input. Sự kiện tác động ± 10 VP hoặc -8 Thẻ lực.
- **Bước 3 (Check Combo):** Duyệt Grid theo từng ngày (cột) để tìm các Tag khớp với bộ quy tắc Combo (src/game/combo.ts): theme (FOOD/CULTURE/ACTION), secondary (INDOOR/OUTDOOR), tag-pair, slot/rhythm, risk/reward. Điểm thưởng được cộng tự động.
- **Bước 4 (Check Distance):** Tính toán khoảng cách giữa các thẻ liền kề. Nếu vượt quá 20km/di chuyển, hệ thống áp dụng điểm phạt. Nếu có ô trống ở giữa, thuật toán sẽ bỏ qua bước kiểm tra này.
- **Bước 5 (Final VP):** Tổng hợp điểm gốc, Bonus Combo và các khoản Penalty để xuất kết quả cuối cùng cho ngày đó.

D. Chuyển đổi Phase (Chưa triển khai)

Tính năng Campaign Progression (chơi nối tiếp nhiều phase) chưa được triển khai. Tuy nhiên, dữ liệu thẻ đã có sẵn cho 3 thành phố: **Sài Gòn (Phase 1)**, **Đà Nẵng (Phase 2)** trong `cards.phase2.ts`, và **Hà Nội (Phase 3)** trong `cards.phase3.ts`. Người chơi có thể chọn thành phố khi bắt đầu game qua `mapSelection.ts`. Kế hoạch mở rộng bản đồ gồm các nhánh Đà Lạt, Nha Trang, Huế được giữ lại cho các phiên bản tương lai.

E. Xuất dữ liệu

Mô tả logic: Hệ thống chuyển đổi toàn bộ Grid thành định dạng Timeline báo cáo sau khi kết thúc 5 ngày.

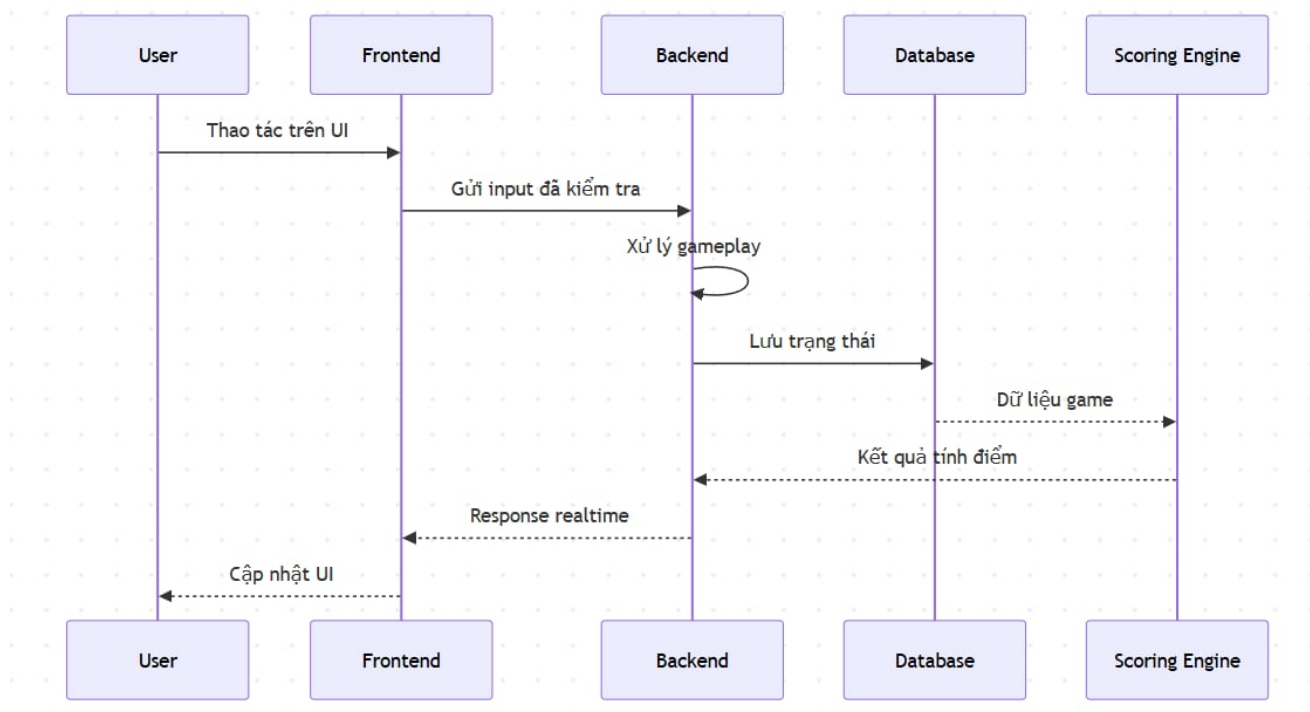
Nội dung Export: Danh sách địa điểm cụ thể, tổng quãng đường di chuyển, dự toán chi phí và chứng chỉ du lịch (`src/export/certificate.ts`). File này giúp người dùng có thể sử dụng như một lịch trình du lịch thực tế.

5.3.2. Luồng dữ liệu

Thành phần tham gia

- Người dùng - Tạo ra các input chính như đăng nhập, chọn thẻ, kéo thả thẻ vào grid, xác nhận vay tài nguyên, chọn tuyến hành trình và chạy mô phỏng.
- Lớp giao diện UI - Chịu trách nhiệm hiển thị:
 - ☐ Màn hình đăng nhập và hồ sơ
 - ☐ Bộ thẻ hiện có, bàn cờ lịch trình 5x5, thanh tài nguyên
 - ☐ Popup xác nhận và cảnh báo
 - ☐ Kết quả mô phỏng, điểm VP và tiến trình phase
- Frontend Logic - Frontend tiếp nhận thao tác từ UI, chuẩn hóa dữ liệu đầu vào, kiểm tra hợp lệ cơ bản và gửi request sang backend. Đồng thời frontend nhận phản hồi để cập nhật giao diện ngay lập tức.
- Backend / Game Engine - Xử lý logic gameplay gồm:
 - ☐ Kiểm tra tài nguyên, kiểm tra điều kiện đặt thẻ
 - ☐ Tính combo và synergy
 - ☐ Thanh tài nguyên
 - ☐ Áp dụng debt token, exhaustion và penalty
 - ☐ Mô phỏng random events, distance check và final VP
- Lưu trữ:
 - ☐ Hồ sơ người chơi: lưu trong PostgreSQL (`server/db.ts`) với bảng `users`, mật khẩu băm PBKDF2. Server vẫn chạy được ở chế độ không-DB nếu chưa cấu hình `DATABASE_URL`.
 - ☐ Trạng thái phòng chơi: sống trong bộ nhớ (in-memory) trên server Node.js.
 - ☐ Lịch sử ván chơi: lưu trong PostgreSQL qua các bảng `matches` và `match_players` (`saveMatchResult`).
 - ☐ Cache tài nguyên PWA qua Service Worker (`sw.js`).

Tổng quan luồng dữ liệu



Hình 5.2: Sơ đồ Data Flow

5.3.3. Luồng dữ liệu theo từng giai đoạn UI/UX

Giai đoạn đăng nhập và truy cập hệ thống

Đây là điểm bắt đầu của luồng dữ liệu. Người dùng nhập email hoặc username cùng password trên màn hình đăng nhập.

- **Input từ UI:**
 - Email hoặc Username
 - Password
- **Xử lý:**
 - Frontend kiểm tra định dạng dữ liệu
 - Backend xác thực tài khoản
 - Database trả về hồ sơ và tiến trình người chơi

- **Output trả về UI:**

- Hồ sơ người chơi
- Lịch sử trận đấu
- Tiến trình tài khoản
- Trạng thái đăng nhập thành công hoặc lỗi xác thực

Giai đoạn Drafting

Mỗi ngày hệ thống cấp 7 thẻ cho người chơi. Người chơi chọn giữ thẻ, chuyển phần còn lại và lặp qua các lượt draft.

- **Input từ UI:**

- Bộ thẻ được phát
- Lựa chọn giữ thẻ
- Hành động bỏ thẻ hoặc xác nhận lượt

- **Xử lý:**

- Frontend ghi nhận thẻ được chọn
- Backend cập nhật pool thẻ, trạng thái lượt và chiến thuật tạm thời
- Nếu thiếu Xu mà vẫn lấy thẻ, hệ thống gán Token Nợ
- Nếu thiếu Thẻ Lực mà vẫn dùng thẻ, hệ thống kích hoạt cơ chế Kiệt Sức và khóa ô tương lai

- **Output trả về UI:**

- Bộ thẻ cuối cùng sau draft
- Trạng thái tài nguyên hiện tại
- Token Nợ nếu phát sinh
- Cảnh báo rủi ro nếu chọn vượt tài nguyên

Giai đoạn xây dựng lịch trình trên Grid 5x5

Người dùng kéo thả thẻ vào grid gồm 5 hàng thời gian và 5 cột ngày để tạo lịch trình.

- **Input từ UI:**
 - Thẻ được chọn
 - Vị trí thả thẻ trên grid
 - Hành động đổi chỗ, hủy thẻ hoặc chốt ngày
- **Xử lý:**
 - Frontend gửi card id, vị trí ô và trạng thái thao tác
 - Backend kiểm tra:
 - Ô có hợp lệ không
 - Tài nguyên có đủ không
 - Thẻ có gây xung đột hoặc vi phạm luật không
 - Combo nào được kích hoạt
 - Khoảng cách di chuyển có vượt ngưỡng không
 - Kết quả được lưu lại theo trạng thái board hiện tại
- **Output trả về UI:**
 - Grid đã cập nhật
 - Combo đang kích hoạt
 - Preview điểm VP tạm thời
 - Tài nguyên còn lại
 - Cảnh báo lỗi nếu đặt sai vị trí

Giai đoạn quản lý tài nguyên

Song song với việc xếp thẻ, hệ thống luôn theo dõi Xu Vàng, Thẻ Lực và các trạng thái phát sinh.

- **Input từ UI:**
 - Hành động mua thẻ
 - Quyết định vay tài nguyên
 - Hành động hủy thẻ để đổi lại tài nguyên

- **Xử lý:**

- Backend kiểm tra số dư tài nguyên
- Nếu vay Xu, hệ thống tạo Token Nợ
- Nếu dùng quá Thẻ Lực, hệ thống sinh trạng thái khóa ô
- Nếu hủy thẻ, hệ thống hoàn lại tài nguyên theo luật
- Thẻ Utility có hiệu ứng đặc biệt (`onPlayEffect`): hồi Xu/Thẻ lực (`RECOVER_XU`, `RECOVER_LA`), miễn phạt khoảng cách (`IGNORE_DISTANCE_NEXT`), giảm giá (`DISCOUNT_XU_NEXT`), nhân đôi VP (`DOUBLE_VP_NEXT`).

- **Output trả về UI:**

- Chỉ số tài nguyên mới
- Trạng thái penalty
- Popup xác nhận hoặc cảnh báo
- Gợi ý hậu quả cho lượt tiếp theo

Giai đoạn Simulation & Scoring

Sau khi người dùng hoàn tất lịch trình, hệ thống khóa grid và chạy chuỗi mô phỏng chấm điểm.

- **Input từ UI / hệ thống:**

- Grid lịch trình cuối cùng
- Dữ liệu tag thẻ
- Tọa độ địa điểm
- Trạng thái Token Nợ, Exhaustion và các penalty khác

- **Xử lý trong backend:**

1. Kiểm tra Token Nợ và trừ điểm tương ứng.
2. Kích hoạt random events theo xác suất 15% dựa trên tag, dùng hàm hash có seed để đảm bảo tái tạo được.
3. Duyệt combo theo hàng và cột để cộng thưởng.

4. Kiểm tra khoảng cách giữa các điểm liên kề, áp dụng phạt nếu vượt 20km.
5. Tổng hợp điểm gốc, bonus và penalty để ra Final VP.

- **Output trả về UI:**

- Tổng điểm VP
- Danh sách combo đạt được
- Các sự kiện ngẫu nhiên đã kích hoạt
- Thống kê quãng đường, chi phí và hiệu quả hành trình

Giai đoạn xuất dữ liệu

Sau mô phỏng, hệ thống có thể chuyển toàn bộ grid thành định dạng timeline phục vụ báo cáo hoặc tái sử dụng như lịch trình du lịch thực tế.

- **Input:**

- Grid cuối cùng
- Kết quả chấm điểm
- Thông tin khoảng cách và chi phí

- **Output:**

- Timeline hành trình
- Danh sách địa điểm
- Tổng quãng đường
- Dự toán chi phí

5.4. Kiểm soát dữ liệu

Hệ thống được thiết kế theo hướng tách biệt rõ dữ liệu lưu trữ tạm thời trong phiên chơi và dữ liệu cần lưu trữ dài hạn nhằm tối ưu hiệu năng, giảm tải backend và đảm bảo an toàn dữ liệu. Với kiến trúc PWA và phần lớn gameplay xử lý ở phía client, nhiều dữ liệu chỉ tồn tại trong thời gian diễn ra ván chơi và không cần đồng bộ liên tục lên server.

5.4.1. Dữ liệu lưu trữ dài hạn

Các dữ liệu được lưu trữ lâu dài bao gồm:

- Bộ dữ liệu thẻ địa điểm (card data), combo, event và content bundle.
- Phiên bản dữ liệu game (**bundle_version**) để đảm bảo tương thích giữa các session.
- Điểm số tổng kết, leaderboard và thống kê gameplay
- Lịch sử ván chơi và timeline hành trình đã hoàn thành

Những dữ liệu này được lưu phía server nhằm đảm bảo tính nhất quán giữa các thiết bị, hỗ trợ cập nhật nội dung game và phục vụ mở rộng hệ thống trong tương lai. Đối với session game, hệ thống lưu **card_id**, **bundle_version** và các thông tin cần thiết để tránh lỗi khi dữ liệu game được cập nhật ở phiên bản mới.

5.4.2. Dữ liệu lưu trữ tạm thời

Các dữ liệu tạm thời chủ yếu tồn tại trong bộ nhớ client hoặc cache cục bộ trong lúc chơi, bao gồm:

- Trạng thái hiện tại của bàn cờ lịch trình 5x5
- Thẻ đang được kéo thả (drag-and-drop)
- Chỉ số Xu Vàng, Thẻ lực và VP tạm thời
- Snapshot trạng thái game theo từng phase
- Kết quả tính điểm trung gian
- Cache tài nguyên PWA như HTML, CSS, JavaScript và hình ảnh

Frontend sử dụng Service Worker và Cache API để lưu đệm tài nguyên nhằm hỗ trợ chế độ offline và giảm thời gian tải lại ứng dụng. Phần lớn logic gameplay cũng chạy trực tiếp trên client để tạo phản hồi gần như tức thời khi người chơi thao tác kéo thả thẻ bài.

5.4.3. Chiến lược lưu trữ dữ liệu

Hệ thống áp dụng chiến lược hybrid storage:

- Client-side storage dùng cho dữ liệu gameplay ngắn hạn và cache offline nhằm giảm độ trễ và hạn chế phụ thuộc mạng.
- Server-side storage dùng cho dữ liệu cần đồng bộ, thống kê và bảo toàn lâu dài.
- Nội dung game được đóng gói theo dạng content bundle có version để hỗ trợ cập nhật từng phần mà không làm hỏng session cũ.
- Dữ liệu gameplay được tổ chức theo schema thống nhất nhằm đảm bảo khả năng mở rộng cho nhiều phase, thành phố và loại thẻ khác nhau.

5.4.4. Biện pháp bảo mật

Hệ thống áp dụng một số biện pháp bảo mật cơ bản phù hợp với phạm vi MVP:

- Runtime Node.js + tsx, kết hợp với cơ chế container Docker (Dockerfile) nhằm hạn chế truy cập file system hoặc network trái phép từ server logic.
- Giao tiếp client-server sử dụng WebSocket và Socket.IO events để kiểm soát luồng message rõ ràng, dễ xác thực và kiểm thử.
- Các session game được gắn version dữ liệu nhằm tránh lỗi không tương thích khi cập nhật nội dung.
- Việc tách logic gameplay theo FSM giúp ngăn các trạng thái game không hợp lệ hoặc thao tác trái quy trình.

5.5. Các kiểu dữ liệu chính (Core Types)

Hệ thống sử dụng một số kiểu dữ liệu nền tảng xuyên suốt cả client và server, được định nghĩa tập trung trong `src/types.ts`:

- **GameBoard** — Ma trận 5×5 , mỗi ô chứa một **PlacedCard** hoặc để trống. Đại diện cho lịch trình $5 \text{ ngày} \times 5 \text{ khung giờ}$.
- **PlacedCard** — Thẻ đã được đặt lên grid, bao gồm: `cardId`, `dayIndex` (0–4), `slotIndex` (0–4: Sáng–Trưa–Chiều–Tối–Khuya), `pos` (tọa độ trong grid).
- **PlayerState** — Trạng thái hiện tại của người chơi: `gold` (Xu Vàng), `stamina` (Thể lực), `debtTokens` (số Token Nợ), `exhaustedSlots` (danh sách ô bị khóa do kiệt sức), `hand` (các thẻ đang giữ).
- **RoomState** — Trạng thái tổng thể của phòng chơi: `phase` ("lobby", "draft", "planning", "simulation", "gameover"), `dayIndex`, `players` (danh sách người chơi), `board` (mảng **GameBoard** cho mỗi người).
- **DraftState** — Trạng thái draft hiện tại: `pool` (các thẻ đang chờ chọn), `pickedCards` (thẻ đã giữ lại), `round` (lượt hiện tại, 1–5), `direction` (chiều rotation, "clockwise" hoặc "counterclockwise").
- **Card** — Thẻ địa điểm: `id`, `name`, `description`, `tags` (mảng tag như "food", "culture"), `lat/lng` (tọa độ), `vp` (điểm cơ bản), `cost` (chi phí Xu), `staminaCost` (tiêu hao Thể lực), `timeSlot` (khung giờ phù hợp).

Các kiểu dữ liệu này được sử dụng chung ở cả client (`app.ts`) lẫn server (`rooms.ts`, `gameEngine.ts`, `draftEngine.ts`), đảm bảo tính nhất quán xuyên suốt hệ thống. Mọi thay đổi về cấu trúc dữ liệu đều được kiểm tra kiểu tĩnh bởi TypeScript compiler trước khi biên dịch.

5.6. Môi trường phát triển & Triển khai

5.6.1. Công cụ phát triển

Thành phần	Công nghệ
Hệ điều hành	Windows 11 / Linux (WSL2)
Ngôn ngữ chính	TypeScript 5.x
CSS Preprocessor	Less (lessc)
Trình biên dịch JS	tsc (TypeScript Compiler)
Server runtime	Node.js 22 + tsx
Trình duyệt đích	Chrome, Firefox, Safari (ưu tiên mobile)
Container	Docker (Alpine Linux)
CI/CD	GitHub Actions
Quản lý phiên bản	Git + GitHub

Bảng 5.7: Công cụ phát triển

5.6.2. Môi trường triển khai

Thành phần	Nền tảng	Cách triển khai
Server (Node.js + Socket.IO)	Hugging Face Spaces	Docker image, push qua GitHub Actions workflow (deploy-server.yml)
Frontend (PWA tĩnh)	GitHub Pages	Build bằng <code>npm run build</code> (tsc + lessc), deploy qua workflow deploy-pages.yml

Bảng 5.8: Môi trường triển khai

5.6.3. Quy trình build

Quy trình build được tự động hóa qua Makefile:

- **make build:** Biên dịch TypeScript (`tsc`) và Less (`lessc`), output vào thư mục `build/`.
- **make server:** Chạy server ở chế độ development (Node.js + tsx, watch mode).
- **make deploy-server:** CI/CD — GitHub Actions build Docker image và push lên Hugging Face Spaces.
- **make deploy-pages:** CI/CD — Build frontend và push lên GitHub Pages.

Frontend được deploy tại: <https://tankhoitv.github.io/comphink-2026/>

6. Tính khả thi của đề án & Rủi ro có thể gặp

6.1. Tại sao dự án của nhóm có thể thực hiện được?

Về mặt kỹ thuật, PoC của dự án có tính khả thi vì cơ chế hiện tại được thu hẹp thành một trò chơi lập lịch trình du lịch ảo, thay vì yêu cầu người dùng phải di chuyển ngoài đời, check-in bằng GPS hay cung cấp nhiều thông tin đầu vào ngay từ đầu. Người dùng chủ yếu tương tác với hệ thống thông qua việc lựa chọn Thẻ Địa Điểm, quản lý tài nguyên như Xu Vàng và Thẻ lực, sắp xếp thẻ vào Bàn Cờ Lịch Trình theo các mốc thời gian, sau đó nhận điểm Hạnh Phúc VP và một lịch trình du lịch hoàn chỉnh sau khi kết thúc ván chơi. Cách tiếp cận này giúp giảm đáng kể độ phức tạp của MVP, vì nhóm có thể tập trung vào vòng lặp gameplay, dữ liệu thẻ và thuật toán tính điểm trước khi mở rộng sang các tính năng ngoài đời như check-in, voucher cho người dùng hoặc dữ liệu thời gian thực.

Kiến trúc PWA cũng phù hợp với phạm vi PoC. Hệ thống có thể tải trước một gói dữ liệu gồm địa điểm, thẻ bài, chỉ số gameplay, combo và sự kiện; sau đó phần lớn logic chơi game được xử lý ở phía client. Điều này giúp trò chơi phản hồi nhanh, không phụ thuộc quá nhiều vào kết nối mạng trong lúc chơi, đồng thời đơn giản hóa vai trò của backend trong giai đoạn đầu. Backend chủ yếu cần phục vụ dữ liệu game, lưu tài khoản hoặc lịch sử chơi nếu có, đồng bộ điểm số và hỗ trợ cập nhật nội dung.

Ngoài ra, các thành phần gameplay có thể được triển khai theo mức độ tăng dần. Ở MVP, nhóm chỉ cần một tập thẻ địa điểm giới hạn, một số loại tài nguyên cơ bản, một bàn cờ lịch trình 3 ngày $\times$ 3 mốc thời gian, cơ chế tính điểm VP, kiểm tra khoảng cách và một vài sự kiện ngẫu nhiên đơn giản. Các chức năng nâng cao như leaderboard hoàn chỉnh, chia sẻ lịch trình, quản lý ưu đãi đối tác, dashboard phân tích hoặc cá nhân hóa lịch trình có thể được giữ lại cho các giai đoạn sau.

6.2. Rủi ro/rào cản có thể gặp phải và hướng xử lý

Vấn đề số 1

Gameplay quá phức tạp so với phạm vi MVP

Mức độ ảnh hưởng: Cao

Phân tích: Cơ chế game gồm nhiều yếu tố như thẻ địa điểm, tài nguyên, combo, khoảng cách,

sự kiện ngẫu nhiên và điểm VP. Nếu triển khai tất cả cùng lúc, nhóm dễ bị quá tải và khó kiểm thử.

Hướng giảm thiểu: MVP chỉ nên giữ vòng lặp cốt lõi: chọn thẻ, xếp thẻ, tiêu Xu/Thẻ lực, tính điểm VP và xuất lịch trình. Các hiệu ứng phức tạp có thể giảm số lượng hoặc để sang giai đoạn sau.

Vấn đề số 2

Cân bằng tài nguyên và điểm số chưa hợp lý

Mức độ ảnh hưởng: Cao

Phân tích: Nếu Xu Vàng, Thẻ lực, chi phí thẻ hoặc điểm VP không cân bằng, người chơi có thể luôn chọn một chiến thuật duy nhất hoặc cảm thấy game quá dễ/quá khó.

Hướng giảm thiểu: Bắt đầu với bộ chỉ số đơn giản; tổ chức playtest; ghi nhận các chỉ số như thẻ được chọn nhiều, điểm trung bình, số lần thiếu tài nguyên và combo phổ biến để điều chỉnh dần.

Vấn đề số 3

Lịch trình cuối game thiếu tính thực tế

Mức độ ảnh hưởng: Cao

Phân tích: Đầu ra của sản phẩm không chỉ là điểm số, mà còn là lịch trình có thể tham khảo ngoài đời. Nếu các địa điểm quá xa nhau, sai thời điểm hoặc thiếu cân bằng giữa ăn uống, tham quan và nghỉ ngơi, giá trị du lịch của sản phẩm sẽ giảm.

Hướng giảm thiểu: Dùng tọa độ và tag địa điểm để kiểm tra khoảng cách; đặt giới hạn phạt khi xếp địa điểm quá xa; phân loại thẻ theo mốc thời gian phù hợp; kiểm thử lịch trình bằng một bộ địa điểm thật ở phạm vi nhỏ.

Vấn đề số 4

Dữ liệu thẻ địa điểm thiếu chính xác

Mức độ ảnh hưởng: Trung bình - Cao

Phân tích: Mỗi Thẻ Địa Điểm đại diện cho một địa điểm thật. Nếu tên, mô tả, tag, vị trí hoặc loại hình hoạt động sai, cả gameplay và lịch trình đầu ra đều bị ảnh hưởng.

Hướng giảm thiểu: Content Manager nên quản lý bộ dữ liệu địa điểm theo schema thống nhất; MVP chỉ dùng một số thành phố/phase giới hạn; ưu tiên dữ liệu đã được kiểm tra thủ công thay vì mở rộng quá nhanh.

Vấn đề số 5

Cập nhật dữ liệu game làm hỏng session cũ

Mức độ ảnh hưởng: Trung bình

Phân tích: Khi thay đổi thông tin thẻ, chỉ số gameplay hoặc combo, các ván chơi cũ có thể không còn tương thích với dữ liệu mới.

Hướng giảm thiểu: Dùng version cho content bundle; lưu `card_id`, `bundle_version` và thông tin cần thiết của session; chỉ áp dụng nội dung mới cho ván chơi mới hoặc có cơ chế migration rõ ràng.

Vấn đề số 6

Leaderboard hoặc điểm số bị gian lận

Mức độ ảnh hưởng: Trung bình

Phân tích: Nếu có bảng xếp hạng, người dùng có thể cố tình chỉnh dữ liệu cục bộ hoặc gửi điểm không hợp lệ lên server.

Hướng giảm thiểu: Trong MVP, leaderboard có thể để ở mức thử nghiệm; nếu đồng bộ điểm, server cần kiểm tra lại kết quả dựa trên log hành động hoặc seed ván chơi thay vì tin hoàn toàn vào điểm do client gửi.

6.3. Kết luận

Nhìn chung, PoC có tính khả thi nếu nhóm giữ đúng phạm vi MVP: xây dựng một trò chơi lập lịch trình du lịch ảo với bộ thẻ địa điểm giới hạn, cơ chế tài nguyên đơn giản, bàn cờ lịch trình rõ ràng, hệ thống tính điểm có thể kiểm thử và đầu ra là một lịch trình du lịch hoàn chỉnh.

Các rủi ro lớn nhất của PoC không nằm ở khả năng xây dựng giao diện hay lưu dữ liệu cơ bản, mà nằm ở ba điểm chính: cân bằng gameplay, chất lượng dữ liệu địa điểm và tính thực tế của lịch trình đầu ra. Nếu nhóm kiểm soát tốt ba yếu tố này, MVP có thể chứng minh được giá trị cốt lõi của sản phẩm: biến quá trình lập kế hoạch du lịch thành một trải nghiệm game thú vị, đồng thời

tạo ra một lịch trình đủ hữu ích để người dùng tham khảo cho chuyến đi ngoài đời.

7. Lộ trình phát triển sản phẩm

Dựa trên kết quả của giai đoạn PoC, nhóm xác định lộ trình phát triển gồm hai giai đoạn kế tiếp nhau, mỗi giai đoạn kéo dài **4 tuần**, tổng cộng **8 tuần**. Giai đoạn đầu tập trung xây dựng vòng lặp gameplay cốt lõi đủ để kiểm chứng giá trị sản phẩm. Giai đoạn hai mở rộng sang trải nghiệm cộng đồng, tiến trình chiến dịch và các công cụ vận hành.

7.1. MVP - Các chức năng cốt lõi & tiêu chí thành công

Mục tiêu giai đoạn (Tuần 1–4): Chứng minh vòng lặp “Play to Plan” hoạt động đầu-cuối. Người chơi phải có thể hoàn thành một ván game đầy đủ — từ lúc bốc thẻ cho đến khi nhận được lịch trình du lịch có thể tham khảo ngoài đời thực — trên một thiết bị di động tầm trung, không cần kết nối mạng liên tục.

7.1.1. Gameplay cốt lõi

- **Giai đoạn Bốc bài (Daily Drafting Phase):** Mỗi ngày hệ thống phát 7 thẻ, người chơi thực hiện 5 lượt chọn-giữ-chuyển. Cơ chế **Token Nợ** (khi thiếu Xu) và **Khóa ô Kiệt sức** (khi cạn Thẻ lực) được áp dụng đầy đủ.
- **Sa bàn lịch trình 5×5:** Lưới 5 ngày × 5 khung giờ (Sáng-Trưa-Chiều-Tối-Khuya), hỗ trợ thao tác kéo thả một mượt trên mobile. Người chơi có thể để trống ô để tạo **Khoảng đệm di chuyển**, hóa giải hình phạt khoảng cách.
- **Hệ thống Tài nguyên Kép:** Thanh Xu Vàng và Thẻ lực hiển thị theo thời gian thực. Cơ chế hủy thẻ để thu hồi tài nguyên được hỗ trợ.
- **Máy tính điểm VP (5 bước):** Kiểm tra nợ → Sự kiện ngẫu nhiên (15%) → Duyệt combo → Phạt khoảng cách (>20km) → Tổng hợp điểm cuối. Toàn bộ pipeline chạy phía server để ngăn chặn gian lận điểm số.
- **Xuất lịch trình du lịch:** Sau mỗi ván, hệ thống chuyển đổi grid thành timeline hành trình bao gồm danh sách địa điểm theo từng ngày, tổng quãng đường di chuyển và dự toán chi phí.

7.1.2. Nội dung & Dữ liệu

- **Bộ Thẻ Địa Điểm:** Hơn 300 thẻ chia làm 3 phase, tổng hợp trong `src/data/cards.all.ts`:
 - ❑ **Phase 1 (Sài Gòn):** Hơn 100 thẻ (`cards.phase1.ts`) — Food, Culture, Action, Utility.
 - ❑ **Phase 2 (Đà Nẵng):** Hơn 100 thẻ (`cards.phase2.ts`) — Food, Culture, Action, Utility.
 - ❑ **Phase 3 (Hà Nội):** Hơn 100 thẻ (`cards.phase3.ts`) — Food, Culture, Action, Utility.

Người chơi chọn thành phố qua màn hình `mapSelection.ts` trước khi vào game. Dữ liệu thẻ được ánh xạ qua `cardMapper.ts`.

- **Hệ thống Combo chi tiết (`src/game/combos.ts`):** Bao gồm 5 nhóm:
 - ❑ **Theme:** 2+ FOOD → +5 VP, 2+ CULTURE → +8 VP, 2+ ACTION → +10 VP.
 - ❑ **Secondary:** 2+ INDOOR → +5 VP, 2+ OUTDOOR → +6 VP, cân bằng trong/ngoài → +4 VP.
 - ❑ **Tag-pair:** 6 cặp (FOOD+CULTURE, ACTION+OUTDOOR, ...) mỗi cặp +4–9 VP.
 - ❑ **Slot/Rhythm:** Có lịch Sáng → +3 VP, có lịch Khuya → +5 VP, dùng 4+ khung → +7 VP.
 - ❑ **Risk/Reward:** Tiết kiệm (ít Xu) → +6 VP, khỏe khoắn (ít Thẻ lực) → +5 VP, liều mạng (chi đậm) → +12 VP.

Combo được tính mỗi ngày dựa trên các thẻ đã đặt trong ngày đó.

7.1.3. Hạ tầng kỹ thuật

- **PWA Mobile-first + Offline:** Service Worker lưu đệm toàn bộ gói dữ liệu thẻ bài. Ván game có thể tiếp tục mà không cần kết nối sau lần tải đầu tiên.

- **Hướng dẫn tương tác (Onboarding Tour):** Module `src/onboarding/` hướng dẫn người chơi mới qua các bước: chọn thẻ draft, kéo thả lên grid, xem kết quả simulation. Có spotlight và tooltip tương tác.
- **Lưu trữ trạng thái ván chơi:** Trạng thái sa bàn, tài nguyên, token nợ, trạng thái khóa ô và điểm VP tạm thời được tự động lưu; làm mới trang sẽ khôi phục đúng tiến trình.
- **Bảng quản trị nội dung (Admin Panel):** Content Manager và Moderator có thể thêm, sửa, xóa thẻ địa điểm và các chỉ số gameplay. Dữ liệu được đánh phiên bản (`bundle_version`) để tránh làm hỏng session đang hoạt động.

7.1.4. Tiêu chí thành công của MVP

- Người chơi hoàn thành được một ván game đầy đủ từ đầu đến cuối mà không gặp lỗi hệ thống.
- Lịch trình xuất ra sau ván game có tính thực tế: các địa điểm liên kế không cách nhau quá 20km trừ khi có ô đệm, khung giờ phân bổ hợp lý.
- Điểm VP được tính đúng và nhất quán giữa client và server.
- Playtest nội bộ xác nhận cân bằng tài nguyên ở mức chấp nhận được — người chơi không bị hết Xu hay Thẻ lực từ lượt đầu tiên trong đa số ván chơi.

7.2. Bản phát hành chính thức

Mục tiêu giai đoạn (Tuần 5–8): Nâng cấp sản phẩm từ bản demo đã kiểm chứng thành một trải nghiệm hoàn chỉnh với tài khoản người dùng, tiến trình chiến dịch nhiều vùng và các tính năng cộng đồng.

7.2.1. Tài khoản người chơi & Lịch sử

- **Xác thực tài khoản (Authentication):** Đăng ký và đăng nhập bằng email/mật khẩu với PBKDF2 + JWT (`server/auth.ts`) đã được triển khai. Tuy nhiên, chưa có chức năng đăng nhập bằng tài khoản mạng xã hội. Hệ thống lưu hồ sơ người chơi trong PostgreSQL (`server/db.ts`) với bảng `users` (`id`, `username`, `display_name`, `password_hash`). Trang hồ sơ cá nhân chi tiết (lịch sử ván đấu, lịch trình đã tạo, thành tựu) chưa được hoàn thiện.

- **Trang hồ sơ cá nhân:** Người chơi xem lại toàn bộ hành trình — các địa điểm đã từng chọn, combo đã kích hoạt và lịch trình đã xuất — như một nhật ký du lịch kỹ thuật số.

7.2.2. Tiến trình chiến dịch & Mở rộng bản đồ

Dữ liệu thẻ đã có sẵn cho Phase 2 (Đà Nẵng) trong `src/data/cards.phase2.ts` và Phase 3 (Hà Nội) trong `src/data/cards.phase3.ts`. Người chơi có thể chọn thành phố qua màn hình `mapSelection.ts` trước khi vào game. Tuy nhiên, Campaign Progression (chơi nối tiếp nhiều phase) và các nhánh bản đồ Đà Lạt, Nha Trang, Huế chưa được triển khai. Sàn bài đã sử dụng lưới 5×5 ngay từ MVP (không cần mở rộng).

7.2.3. Tính năng cộng đồng & Thi đua

- **Bảng xếp hạng (Leaderboard):** Đã triển khai — hiển thị điểm VP theo ngày chơi (xem trong `src/app.ts`). Tuy nhiên chưa có xếp hạng toàn cầu hoặc theo nhóm bạn bè.
- **Chia sẻ lịch trình:** Người chơi xuất bản lịch trình hoàn chỉnh dưới dạng file pdf, hiển thị điểm VP, combo đã kích hoạt, danh sách địa điểm và kế hoạch từng ngày. Cho phép người chơi tham khảo lịch trình của nhau.

7.2.4. Tiêu chí thành công của bản phát hành chính thức

- Người chơi có tài khoản có thể xem lại đầy đủ lịch sử ván chơi và lịch trình đã tạo qua nhiều phiên đăng nhập khác nhau.
- Bảng xếp hạng không bị thao túng bởi điểm gian lận sau khi cơ chế xác thực phía server được bật.

8. Các quyết định kỹ thuật

8.1. Công nghệ thực hiện (Tech Stack)

8.1.1. Backend

Tech stack thực tế

Kiến trúc tuân theo sơ đồ phân chia ba panel: **Build pipeline** (Makefile), **Client** (Trình duyệt), và **Server** (Deno), với **Logic dùng chung** (src/) là nhân domain.

Tầng	Công nghệ	Vai trò trong Backend	Lý do chọn
Runtime	Node.js + tsx	Chạy <code>server/index.ts</code> — điểm vào HTTP + Socket.IO listener	Hệ sinh thái npm phong phú (Socket.IO, tsx), team quen thuộc với Node.js, dễ tìm tài liệu
Transport	Socket.IO	Kênh real-time hai chiều từ <code>socketClient.ts</code> (client) đến <code>index.ts</code> (server)	Tự động fallback transport, built-in room management, dễ dàng broadcast, type-safe events
Messaging	Socket.IO events (<code>types.ts</code>)	Giao thức event-based với kiểu TypeScript (<code>ClientToServerEvents</code> / <code>ServerToClientEvents</code>)	An toàn kiểu dữ liệu, dễ debug, không cần tự implement message framing
State Machine	<code>rooms.ts</code> (phase-based state machine)	State machine phòng — phase: lobby → cinematic → draft → planning → simulation → result → gameover, dayIndex, player resources	Vòng lặp game có trình tự pha rõ ràng (5 ngày × 4 phase); kết hợp <code>gameEngine.ts</code> , <code>draftEngine.ts</code> , <code>timerEngine.ts</code>

Room Logic	<code>gameEngine.ts</code>	Game state factories: board, deck, player, view projections	Tách biệt logic game khỏi life-cycle phòng
Draft Engine	<code>draftEngine.ts</code>	Draft state machine: pool creation, rotation, pick/confirm	Tách biệt logic draft khỏi game engine
Timer Engine	<code>timerEngine.ts</code>	Simulation tick: debt scan, random events, combo, distance, day transitions	Xử lý simulation theo thời gian thực
Auth	<code>auth.ts</code> (PBKDF2 + JWT)	Đăng ký, đăng nhập, xác thực token (<code>crypto.subtle</code> , không external deps)	Bảo mật cơ bản, không phụ thuộc thư viện ngoài, dễ maintain
Types	<code>types.ts</code>	Định nghĩa kiểu Socket.IO event (client ↔ server contract)	Đảm bảo an toàn kiểu dữ liệu xuyên suốt
Triển khai	Docker (Dockerfile) — Hugging Face Spaces	Đóng gói Node.js runtime + tài nguyên vào container image	Loại bỏ drift môi trường; tích hợp sẵn với Hugging Face Spaces + GitHub Actions (<code>deploy-server.yml</code>)

8.1.2. Frontend

1. Tech Stack Thực Tế

Kiến trúc tuân theo sơ đồ phân chia ba panel: **Build pipeline (Makefile)**, **Client (Trình duyệt)**, và **Server (Node.js + tsx)**, với **Logic dùng chung (src/)** là nhân domain.

Tầng	Công nghệ	Vai trò trong Frontend	Lý do chọn
Runtime	Browser (DOM API)	Môi trường thực thi trực tiếp <code>app.js</code> và render <code>index.html</code> , <code>client.css</code>	Môi trường đích bắt buộc của Web App; hỗ trợ native HTML5 Drag and Drop API cho cơ chế xếp thẻ
Ngôn ngữ	TypeScript & Less	Định nghĩa logic điều khiển UI an toàn (TS); hệ thống biến và lồng ghép cho giao diện (Less)	Bắt lỗi tương tác DOM từ lúc code; quản lý màu sắc thẻ bài (Đà Lạt, Sài Gòn) dễ dàng nhờ biến Less
Giao diện & Cấu trúc	HTML5 & CSS Grid	Xây dựng khung lưới lịch trình 5x5 và quản lý layout tổng thể	CSS Grid ánh xạ hoàn hảo 1:1 với logic ma trận 3 ngày × 5 slot thời gian trong <code>board.ts</code>
Transport	WebSocket (Browser API)	Kênh real-time kết nối lên server để truyền nhận hành động người chơi	Cần gửi tọa độ thẻ bài (drag-drop) lập tức không cần tải lại trang; nhận broadcast tức thời từ đối thủ
Trạng thái UI	Vanilla TS (<code>app.ts</code>)	Quản lý DOM state: cập nhật thanh năng lượng, số Xu, vẽ lại lưới 5x5 khi thẻ được thả	Giữ bundle size cực nhỏ, không có overhead của Virtual DOM (như React/Vue) cho một game ưu tiên tốc độ

Tầng	Công nghệ	Vai trò trong Frontend	Lý do chọn
Domain Logic	Tái sử dụng <code>src/</code>	Chạy bộ luật (<code>rules.ts</code> , <code>score.ts</code>) trực tiếp trên trình duyệt để tính điểm tạm thời (Optimistic UI)	Tính điểm và báo lỗi sai luật ngay khi người chơi đang cầm thẻ kéo thả, không cần đợi server phản hồi
Build Tool	tsc (TypeScript compiler)	Biên dịch TypeScript sang JavaScript, kiểm tra kiểu tĩnh	Đơn giản, không cần bundler cho MVP; giảm độ phức tạp của build chain; output là các file .js riêng lẻ trong <code>build/</code>
Offline PWA	Service Worker (Cache API)	Lưu đệm <code>index.html</code> , <code>app.ts</code> , <code>client.css</code> và hình ảnh trong <code>assets/</code>	Đảm bảo người chơi mở được game ngay cả khi mất mạng (đang đi xe khách, leo núi); giảm tải băng thông

8.2. Các quyết định kiến trúc

8.2.1. Backend

ADR-001 — Sử Dụng Node.js + tsx Làm Server Runtime

Bối cảnh: Game server cần chạy TypeScript, expose HTTP endpoint và WebSocket, deploy trong Docker container. Ban đầu đề xuất Deno nhưng trong quá trình phát triển, team nhận thấy Node.js đã có hệ sinh thái hơn cho Socket.IO, dễ tìm tài liệu, và tsx cho phép chạy TypeScript trực tiếp không cần build riêng.

Quyết định: Sử dụng Node.js + tsx làm runtime server chính.

Lý do: Socket.IO yêu cầu npm package — Node.js có sẵn support. tsx cho phép chạy TypeScript trực tiếp (type-stripping) tương tự Deno. Team đã có kinh nghiệm với Node.js. Dockerfile chỉ cần `FROM node:22-alpine`.

Hệ quả: Phải cài npm packages. Không có sandbox bảo mật tích hợp như Deno, thay vào đó

dùng container Docker để cô lập.

ADR-002 — Sử Dụng Socket.IO Cho Messaging Client-Server

Bối cảnh: Pha Bốc Bài yêu cầu đồng bộ lượt bài và truyền bài giữa các người chơi. Pha Mô Phỏng cần server push sự kiện đến tất cả clients.

Quyết định: Socket.IO transport với event-based messaging. Server định nghĩa `ClientToServerEvents` và `ServerToClientEvents` trong `types.ts`.

Lý do: Socket.IO cung cấp sẵn cơ chế room, broadcast, và fallback transport. Event-based approach dễ hiểu hơn JSON-RPC. Type safety được đảm bảo qua TypeScript generics.

Hệ quả: Client phải load thư viện `socket.io.min.js` từ CDN. Server dùng `socket.io` package. Cả client và server đều tham chiếu đến cùng kiểu event trong `types.ts`.

ADR-003 — Implement Game Loop Bằng Phase-based State Machine Trong rooms.ts, Kết Hợp Các Engine Chuyên Biệt

Bối cảnh: Game có vòng lặp với trình tự phase nghiêm ngặt: Lobby → Draft → Planning → Simulation, lặp qua 5 ngày.

Quyết định: Implement state machine trong `rooms.ts` với `phase` field (kiểu string enum: `"lobby"`, `"cinematic"`, `"draft"`, `"planning"`, `"simulation"`, `"result"`, `"gameover"`) và `dayIndex`. Các logic chuyên biệt được tách ra: `gameEngine.ts` (khởi tạo state), `draftEngine.ts` (draft rotation), `timerEngine.ts` (simulation).

Lý do: FSM tường minh với transition function là phù hợp nhưng team chọn cách đơn giản hơn: kiểm tra phase hiện tại trước khi xử lý action. Điều này đủ để ngăn trạng thái game không hợp lệ.

Hệ quả: Mỗi action handler trong `rooms.ts` đều kiểm tra `state.phase` trước khi xử lý. Các engine phụ (`draftEngine.ts`, `timerEngine.ts`) query `state.phase` hiện tại.

ADR-004 - Giữ Domain Logic Trong src/ Dùng Chung Biên Dịch Cho Cả Client Và Server

Bối cảnh: Các quy tắc như tương thích tag-slot, tính phạt khoảng cách, và tính điểm VP cần thiết cả trên client (xem trước real-time) lẫn server (kiểm tra chuẩn quyền). Nhân đôi code quy tắc tạo ra nguy cơ drift.

Quyết định: Đặt tất cả rule logic (`board.ts`, `rules.ts`, `score.ts`, `dice.ts`) trong `src/` và biên dịch vào cả client bundle (qua `tsc`) và import graph của Node.js server.

Lý do: Optimistic UI với server authority — UX nhanh + ngăn gian lận. Các module TypeScript thuần không có side effects có thể được import bởi cả hai target.

Hệ quả: Tất cả module dùng chung phải không chứa platform-specific API. Bất kỳ module nào cần Deno (file I/O, crypto) hoặc DOM (`window`, `document`) phải được tách thành platform adapter.

ADR-005 — Sử Dụng Docker để Đóng Gói Server và Triển Khai lên Hugging Face Spaces

Bối cảnh: Deno server phải có thể triển khai lên hạ tầng cloud. Team đã xem xét ba mô hình triển khai cho lần ra mắt đầu tiên.

Quyết định: Đóng gói server dưới dạng Docker image sử dụng `Dockerfile` trong panel Server.

Lý do: WebSocket session là stateful — FSM `game.ts` của phòng sống trong bộ nhớ server suốt phiên. Serverless functions không thể giữ in-memory state qua các lần gọi mà không có external store (Redis, v.v.), điều này thêm độ trễ và chi phí ở giai đoạn MVP.

Hệ quả: Mở rộng ngang yêu cầu sticky sessions (định tuyến mỗi WebSocket theo room ID đến cùng container) hoặc migrate room state sang external store. Đây là nút thắt cổ chai mở rộng chính cần xem xét lại nếu số lượng phòng đồng thời vượt quá khả năng của một container.

Triển khai thực tế:

- **Server:** Docker image được triển khai lên Hugging Face Spaces qua GitHub Actions workflow (`deploy-server.yml`). Workflow được kích hoạt khi push vào nhánh main có thay đổi trong `server/`, `src/data/`, `src/types.ts`, hoặc `Dockerfile`.
- **Frontend:** Static PWA được triển khai lên GitHub Pages qua workflow `deploy-pages.yml`. Build bằng `npm run build` (`tsc + lessc`), output được đẩy lên GitHub Pages.

8.2.2. Frontend

ADR-001 - Sử Dụng Vanilla TypeScript Thay Vì Framework (React/Vue)

Bối cảnh: Game yêu cầu các tương tác kéo thả (Drag and Drop) phức tạp, hiệu năng cao trên lưới 5x5. Kích thước file tải xuống (bundle size) cần cực kỳ tối ưu vì target là người dùng di động, PWA, có thể sử dụng mạng 3G/4G yếu.

Quyết định: Viết logic UI hoàn toàn bằng Vanilla TypeScript, không dùng Framework UI.

Lý do: Bản chất của game là thao tác trực tiếp lên tọa độ lưới. Việc sử dụng HTML5 Drag/Drop native nhanh và nhẹ hơn nhiều so với việc wrap qua Virtual DOM. Bundle size sẽ giảm được hàng trăm KB, giúp game tải gần như lập tức.

Hệ quả: Developer phải tự quản lý việc đồng bộ giữa State (dữ liệu) và View (DOM). Mọi thay đổi dữ liệu phải đi kèm hàm `updateUI()` thủ công.

ADR-002 - Xử Lý Giao Diện Bằng CSS Preprocessor (Less) Kết Hợp CSS Grid

Bối cảnh: Giao diện yêu cầu một bàn cờ thời gian 5x5, các thẻ bài có màu sắc riêng theo khu vực, và cần khả năng bảo trì cao khi thêm các vùng du lịch mới.

Quyết định: Sử dụng Less để biên dịch ra CSS, và dùng CSS Grid làm layout chính cho board game.

Lý do: CSS Grid (`grid-template-columns: repeat(5, 1fr)`) sinh ra là để làm lưới 5x5. Kết hợp với biến của Less, việc thay đổi theme màu sắc hoặc kích thước toàn bộ game chỉ cần sửa ở 1 nơi duy nhất thay vì tìm-thay-thế hàng loạt class.

Hệ quả: Dây chuyền Makefile cần thêm bước gọi `lessc`. Không dùng trực tiếp file `.css` trong thư mục source mà phải làm việc qua `client.css` ở thư mục `build/`.

ADR-003 - Triển Khai Optimistic UI Cho Phép Tương Tác Không Độ Trễ

Bối cảnh: Game chơi qua mạng (WebSocket). Khi người chơi kéo thẻ thả vào lưới, nếu phải đợi server xác nhận “hợp lệ” rồi mới vẽ vào lưới thì sẽ bị khựng (lag), gây trải nghiệm thao tác kém mượt mà.

Quyết định: Import thư mục `src/` (`rules.ts`, `score.ts`) vào `app.ts`. Tính toán điểm tạm thời và check luật trực tiếp ngay khi thả thẻ bài. Giả định thành công, sau đó gửi sự kiện lên server

qua `socketClient.ts`.

Lý do: Vừa mượt (do xử lý tức thời ở Browser) vừa an toàn (do server vẫn cầm trịch kết quả cuối). Vì dùng chung logic TypeScript, client không bao giờ báo hợp lệ cho một nước đi mà server sẽ từ chối.

Hệ quả: tsc phải gom mã nguồn thư mục `src/` vào `app.js`. Client phải chuẩn bị cơ chế “rollback” (hủy thao tác, giật thẻ bài về vị trí cũ) nếu Server bị lỗi do mạng bất đồng bộ.

ADR-004 - Tối Ưu Tải Trạng Thái Bằng Service Worker (sw.js) & Cache Storage

Bối cảnh: Là game du lịch, người chơi có thể mở game khi đang di chuyển trên tàu xe, nơi kết nối mạng chập chờn. Việc phải fetch lại đồng ảnh thẻ bài `img/` liên tục sẽ làm nghẽn game.

Quyết định: Sử dụng Service Worker (`sw.js`) để cache `index.html`, `app.js`, `types.js`, và các file `.js` khác trong `build/`, `client.css` và toàn bộ thư mục `assets/`.

Lý do: Chiến lược “Cache-First” sẽ giúp game load mất < 1 giây ở các lần mở sau, giảm tải server. Đặc biệt quan trọng cho các file ảnh tĩnh không bao giờ thay đổi (như mặt thẻ bài).

Hệ quả: Developer phải xử lý cẩn thận phiên bản (versioning) trong `sw.js`. Nếu build bản cập nhật mới (ví dụ sửa lỗi tính điểm) mà không đổi key cache, người chơi sẽ kẹt ở phiên bản lỗi vĩnh viễn dù đã có mạng.